

1. Description de la méthode :

a. Prototype de la méthode

```
private bool VerificationSuperpositionZone(GMapPolygon previous_zone,  
GMapPolygon zone)
```

b. Description

Cette méthode vérifie si il existe une collision entre les deux polygone passé en paramètre en utilisant le théorème de l'axe séparé (SAT)

c. Paramètre d'entrée

Paramètres d'entrés	Type	Description
previous_zone	GMapPolygon	Une zone sur la carte
zone	GMapPolygon	Une autre zone sur la carte

d. Sortie

La méthode retourne true si il y a une collision sinon elle retourne false

2. Explication du SAT :

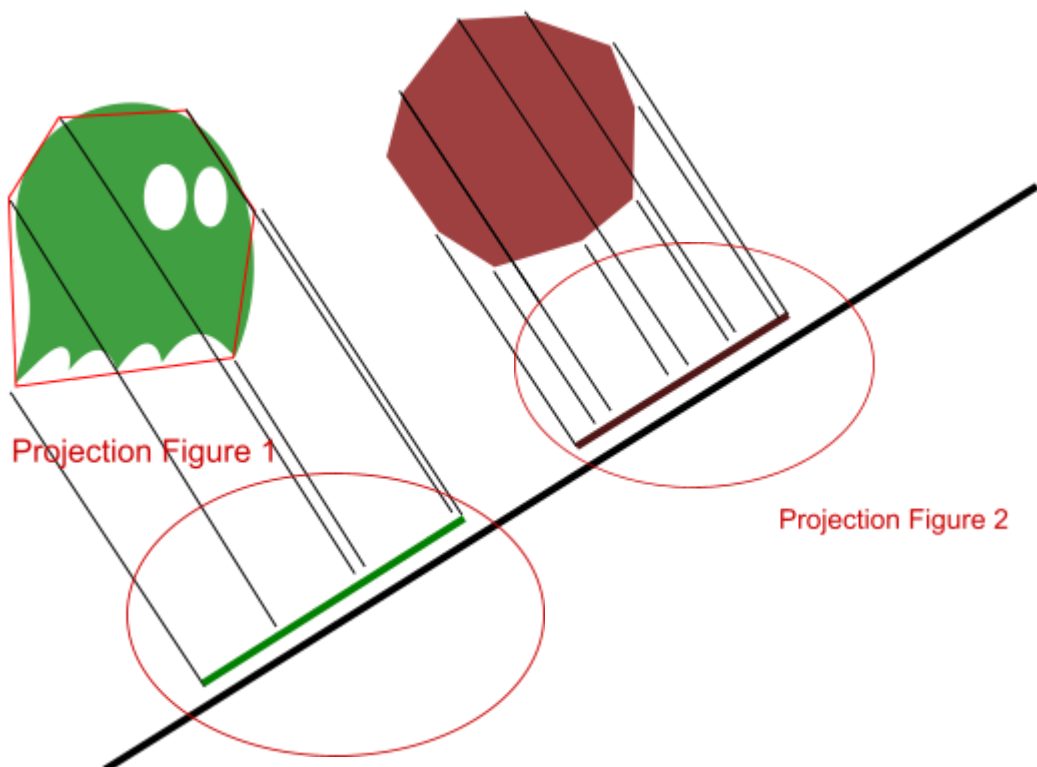
2.1. Prérequis :

Afin de pouvoir utiliser le théorème de l'axe séparé il faut que les deux formes testé soit :

- convexe (figure ou tout les angles sont inferieur a 180° :
<https://graphicmaths.com/img/gcse/geometry/convex-concave-polygon.png>)
- sur un plan en 2 dimension(x, y)

2.2. Application du SAT

Le but du SAT va être de projeter chaque sommets sur un axe de projection afin de vérifier que les deux projections ne se superpose pas comme ceci :

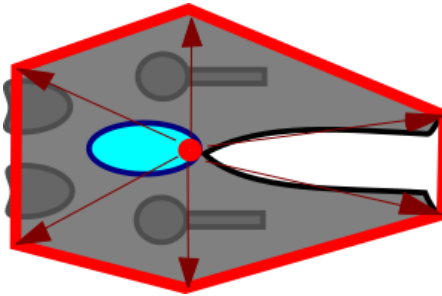


Afin d'appliquer le SAT, il va falloir suivre 3 étapes :

2.2.1. Création des vecteurs à partir du centre de la figure

Afin de projeter nos sommets sur l'axe de projection on a deux choix soit :

- On créer des vecteurs a partir du centre de la figure jusqu'à chacun de ses sommets comme ceci :



- On projette directement les sommets sur l'axe de projections (si on choisit cette méthode alors on a rien a créer)

La création des vecteurs va être gérer par cette partie du code:

```
//Creation des vecteurs a partir du centre de la figure jusqu'à chaque sommet de celle ci
list_previous_zone = ZoneToVector2(previous_zone);
list_zone = ZoneToVector2(zone);
```

et va dépendre de cette méthode :

```
//Methode creeant des vecteurs entre le centre de la zone passé en paramètre et chaque sommets de cette zone
2 references
private List<Vector2> ZoneToVector2(GMapPolygon zone)
{
    List<Vector2> vecteurs = new List<Vector2>();
    var min_y = zone.Points.Min(y => y.Lat);
    var max_y = zone.Points.Max(y => y.Lat);
    var min_x = zone.Points.Min(x => x.Lng);
    var max_x = zone.Points.Max(x => x.Lng);
    var centre = LatLngToPoint(new PointLatLng(min_y + ((max_y - min_y)/2), min_x + ((max_x - min_x)/2))); //Projection de chacun des points sur un plan 2D

    //Creation des vecteurs
    for (int i = 0; i < zone.Points.Count; i++)
    {
        Vector2 vecteur = new Vector2(Convert.ToSingle(LatLngToPoint(zone.Points[i]).X - centre.X), Convert.ToSingle(LatLngToPoint(zone.Points[i]).Y - centre.Y));
        vecteurs.Add(vecteur);
    }

    return vecteurs;
}
```

2.2.2. Choix de l'axe de projection

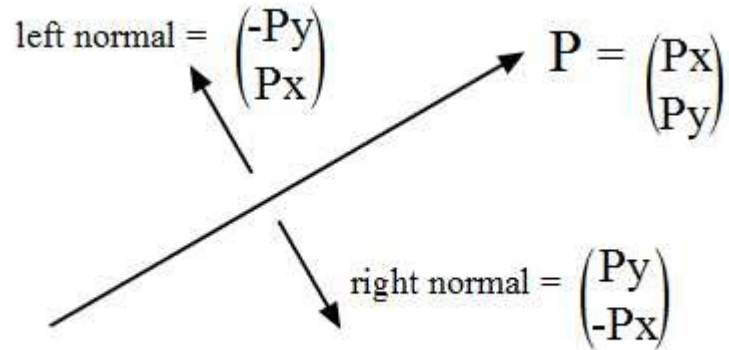
Comme expliqué plus tôt le but du SAT est de vérifier si les deux projections des deux figures sur l'axe de projection ne se superpose pas, si cela est vrai alors cette axe sera appelé axe de séparation et dans ce cas il n'y a aucune superposition entre les deux figures

en générales vérifié un seul axe n'est pas suffisant car dans les cas où les formes sont penché cela vérifiera mal la superposition des deux

c'est pour cela que pour un résultat plus précis on va vérifié par rapport à la normale de chaque arête des deux figures

Documentation méthode *VerificationSuperpositionZone*

Une normale est un vecteur unitaire (de longueur 1) perpendiculaire au vecteur directeur de l'arête :



Dans notre méthode la création des normales va être fait par cette partie du code :

```
//Calcule des normales de chaque vecteur
normale_previous_zone = CreationNormales(list_previous_zone);
normale_zone = CreationNormales(list_zone);
```

et va être réalisé à l'aide de cette méthode :

```
2 références
private List<Vector2> CreationNormales(List<Vector2> arete)
{
    List<Vector2> normales = new List<Vector2>();

    foreach (Vector2 vecteur in arete)
    {
        Vector2 normale = new Vector2(vecteur.X, -vecteur.Y);
        normales.Add(normale);
    }
    return normales;
}
```

2.2.3. *Projection les points sur les axes de projections*

Pour projeter les points sur les axes on a deux manière en fonction de quelle méthode on a choisi pour l'étape 1, soit :

- On projette nos vecteurs sur l'axe est dans ce cas la on fait un produit scalaire de la normales et du vecteur du sommet
- On projette directement les points sur l'axe

Documentation méthode *VerificationSuperpositionZone*

Dans notre méthode, cette partie va être gérée par ce bout de code :

```
//Projection des vecteurs sur les normales de chaque polygone
foreach (Vector2 normale in normale_previous_zone)
{
    resultat_previous_zone = ProjectionVecteurAxe(list_previous_zone, normale);
    resultat_zone = ProjectionVecteurAxe(list_zone, normale);

    //aucune collision si le max de la première zone est inférieur au min de la deuxième et inversement
    if (resultat_previous_zone.Max() < resultat_zone.Min() || resultat_zone.Max() < resultat_previous_zone.Min())
    {
        is_valide = false;
        break;
    }
}
```

Et va dépendre de cette méthode :

```
//Methode réalisant un produit scalaire entre les vecteurs et l'axe passé en paramètres//
4 références
private List<float> ProjectionVecteurAxe(List<Vector2> vecteurs, Vector2 axe)
{
    List<float> resultat = new List<float>();
    foreach (Vector2 vecteur in vecteurs)
    {
        resultat.Add(Vector2.Dot(vecteur, axe));
    }
    return resultat;
}
```

2.3. Vérification de la superposition

Si la valeur maximum des produits scalaire fait sur les normales de la figure 1 est inférieure à la valeur minimum des produits scalaire fait sur les normales de la figure 2 ou inversement alors l'axe est séparateur

Dans notre méthode cette vérification se fait avec ce bout de code :

```
287 //Projection des vecteurs sur les normales du premier polygone
288 foreach (Vector2 normale in normale_previous_zone)
289 {
290     resultat_previous_zone = ProjectionVecteurAxe(list_previous_zone, normale);
291     resultat_zone = ProjectionVecteurAxe(list_zone, normale);
292
293     //aucune collision si le max de la première zone est inférieur au min de la deuxième et inversement
294     if (resultat_previous_zone.Max() < resultat_zone.Min() || resultat_zone.Max() < resultat_previous_zone.Min())
295     {
296         is_valide = false;
297         break;
298     }
299 }
300
301
302 if (is_valide == true)
303 {
304     //Projection des vecteurs sur les normales du deuxième polygone
305     foreach (Vector2 normale in normale_zone)
306     {
307         resultat_previous_zone = ProjectionVecteurAxe(list_previous_zone, normale);
308         resultat_zone = ProjectionVecteurAxe(list_zone, normale);
309
310         if (resultat_previous_zone.Max() < resultat_zone.Min() || resultat_zone.Max() < resultat_previous_zone.Min())
311         {
312             is_valide = false;
313             break;
314         }
315     }
316 }
317
318 return is_valide;
319 }
320
321 }
```

3. Source :

Explication théorème de l'axe séparé :

<https://textbooks.cs.ksu.edu/cis580/04-collisions/04-separating-axis-theorem/index.html>